



Parqueadero Boyacá

Desarrollo

Aprendiz: Eduar Danilo Paipilla Zorro. Ficha: 3171062.

PARTE A — Análisis de Código

Pregunta 1: Identifique las capas de la arquitectura del sistema

Respuesta

El sistema tiene una arquitectura cliente-servidor dividida en dos capas principales:

Capa	Tecnología	Puerto	Responsabilidad
Frontend	PHP 8 + Apache XAMPP	Puerto 80	Mostrar la interfaz gráfica, formularios HTML, estilos CSS, ejecutar JavaScript y consumir la API Java
Backend	Java 8 + Servlets + Apache Tomcat	Puerto 8080	Procesar lógica de negocio, manejar peticiones HTTP REST, conectarse a MySQL y retornar JSON

Explicación

El frontend funciona como cliente.

El usuario interactúa con páginas como:

- index.php
- entrada.php
- salida.php
- historial.php

Estas páginas llaman al backend usando cURL y HTTP.

El backend funciona como servidor REST.

Las clases Servlet reciben las peticiones:

- VehiculoServlet.java
- RegistroServlet.java

Luego llaman a los DAO para consultar o modificar la base de datos MySQL.

Pregunta 2: Analice la clase ConexionDB.java

Respuesta

La clase ConexionDB.java utiliza la url, usuario y clave para crear la conexión usando el driver JDBC y DriverManager.getConnection(URL, USUARIO, CLAVE) creando una conexión nueva usando DriverManager no existe un pool de conexiones ni una conexión compartida.

Pregunta 3: Explique el patrón DAO usando VehiculoDAO como ejemplo

Respuesta

El patrón DAO (Data Access Object) separa el acceso a la base de datos de la lógica de negocio.

En la clase:

VehiculoDAO.java

Esta se encarga exclusivamente de ejecutar operaciones SQL sobre la tabla vehiculos como insertar, listarTodos, buscarPorId, buscarPorPlaca, actualizar y eliminar.

Evitando manejar logica de negocio o metodos http, lo que busca este patron es hacer un orden de responsabilidades, para acceso a datos, logica de negocio y metodos http esto en busqueda de un mejor orden facilitando la escalabilidad y manejo del codigo.

Pregunta 4: Analice registrarSalida() en RegistroDAO.java

El metodo maneja las salidas de vehiculos del estacionamiento validando la hora de entrada y el estado de la entrada, siendo el valor minimo cobrable de 1 hora, validando tiempos de ≤ 60 min se cobra una hora > 60 min se cobra el numero de horas correspondientes a los minutos , aproximando al mayor siendo el caso que si fueran 61 minutos ya se cobra 2 horas, el metodo hace uso de UPDATE en consulta sql modificando el registro de entrada modificando sus valores a un estado ya finalizado.

Pregunta 5: Analice apiRequest() en config.php

Respuesta

El frontend PHP usa cURL para comunicarse con la API Java REST este permite el tratamiento de datos mandados desde back asi como la conexion con la API asi evitando que se muestren directamente los datos, y sean procesados de forma debida.

PARTE B — Flujo de Datos

Actividad B.1 : Flujo Registro de Entrada

Respuesta

Usuario abre entrada.php en el navegador



Usuario busca placa BOY004



entrada.php llama:

GET /api/vehiculos?placa=BOY004



config.php usa cURL para enviar petición HTTP al backend Java



VehiculoServlet recibe la petición GET



VehiculoDAO.buscarPorPlaca("BOY004")



MySQL consulta tabla vehiculos



No existe vehículo → retorna 404 o vacío



PHP muestra formulario para crear vehículo



Usuario llena:

placa, tipo, propietario, telefono



PHP envía:

POST /api/vehiculos

JSON:

{

"placa": "BOY004",

"tipo": "CARRO",

```
"propietario":"Juan",  
"telefono":"3100000000"  
}
```

↓

VehiculoServlet recibe POST

↓

VehiculoDAO.insertar()

↓

MySQL inserta vehículo

↓

Backend responde JSON del vehículo creado

↓

PHP registra entrada:

POST /api/registros

```
{  
  "vehiculoId": 4  
}
```

↓

RegistroServlet recibe petición

↓

RegistroDAO.registrarEntrada()

↓

MySQL inserta registro ACTIVO

↓

Backend responde:

```
{  
  "id":15,
```

"estado":"ACTIVO"

}

↓

PHP muestra mensaje:

"Entrada registrada correctamente"

Actividad B.2 — Tabla HTTP

Acción	Método HTTP	URL Endpoint	Cuerpo JSON	Respuesta esperada
Listar vehículos	GET	/api/vehiculos	No aplica	Array JSON de vehículos
Buscar placa BOY002	GET	/api/vehiculos? placa=BOY002	No aplica	JSON vehículo
Registrar vehículo nuevo	POST	/api/vehiculos	{"placa":"BOY010", "tipo":"MOTO", "propietario":"Luis", "telefono":"3200000000"}	201 Created
Actualizar teléfono ID=1	PUT	/api/vehiculos/1	{"telefono":"3119999999"}	JSON actualizado
Registrar entrada ID=2	POST	/api/registros	{"vehiculoId":2}	Registro ACTIVO
Registrar salida ID=5	PUT	/api/registros/5/salida	No aplica	Tarifa calculada
Ver historial finalizado	GET	/api/registros? estado=FINALIZADO	No aplica	Array JSON
Eliminar vehículo ID=3	DELETE	/api/vehiculos/3	No aplica	200 OK

PARTE C - Ampliación del Sistema (40 puntos)

Respuesta

Se implementaron las funciones adicionales 1 y 2 (Reporte de Ingresos del Dia y Búsqueda Avanzada en el Historial) a continuación se anexan enlaces a documentos modificados y especificaciones:

Opción 1: Reporte de Ingresos del Dia

Se modificaron las clases RegistroDAO.java creación de método ObtenerIngresoPorDia() el cual usa una consulta SQL de filtración para traer la cantidad de vehículos procesados en ese día y el valor total recaudado, además de la creación de la clase RegistroDTO.java en la cual se almacena la información recopilada de dicho método para posteriormente serializarla a formato json para la respuesta del Servlet en el cual se modifico el método doGet() para el mapeo de la url para reportes. php.

[RegistroDAO.java](#) ← Click aqui para ver

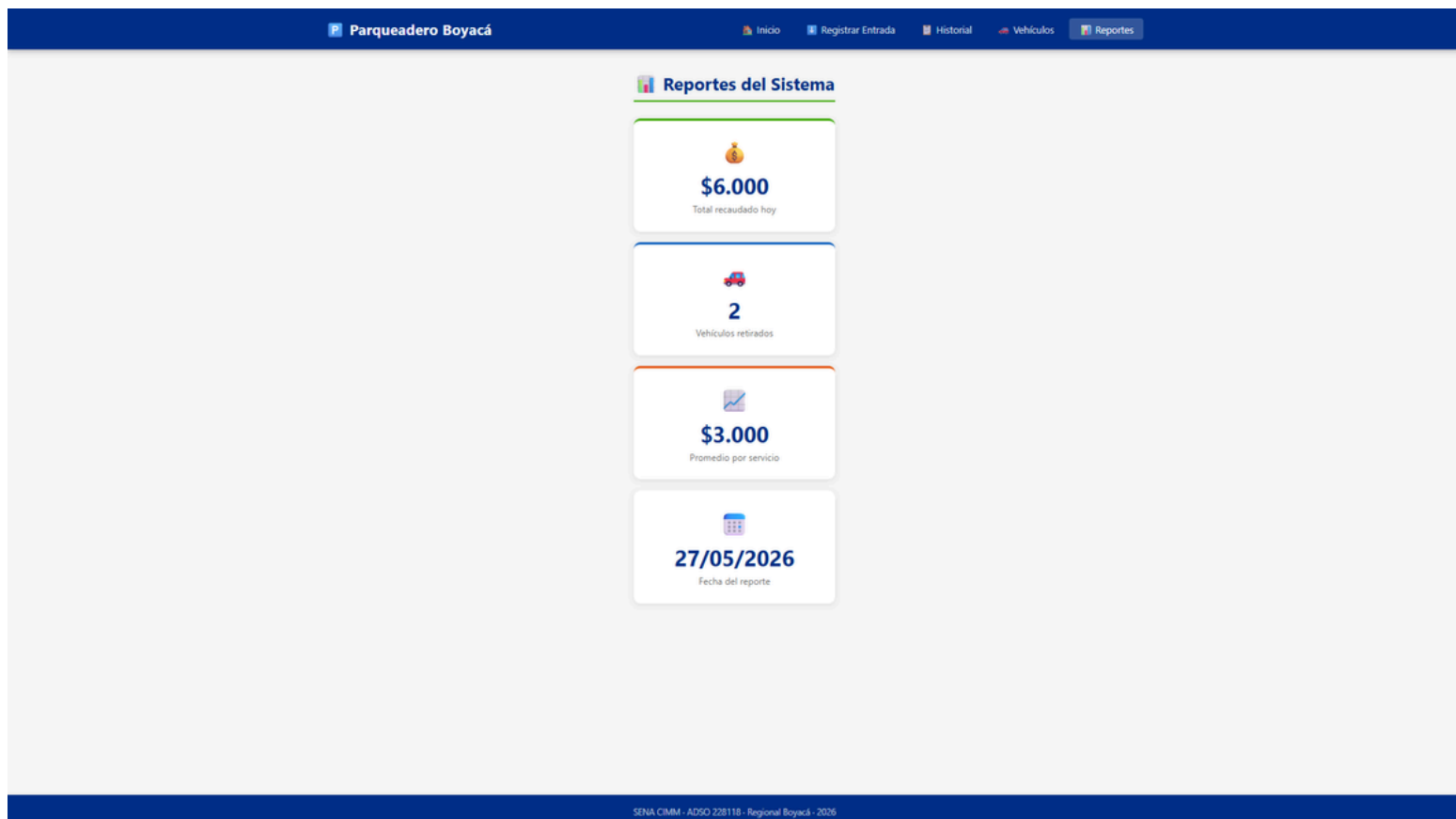
[RegistroDTO.java](#) ← Click aqui para ver

[RegistroServlet.java](#) ← Click aqui para ver

[reportes.php](#) ← Click aqui para ver

Imágenes - función

vista-función-ingresos Diarios



método-función-ingresos Diarios

```

89 public RegistroDTO ObtenerIngresoPorDia() throws SQLException { 1 usage
90     RegistroDTO registroDto = new RegistroDTO();
91
92     String sql = "select sum(tarifa) as totalDia, count(*) as cantidadSalidas from registros where DATE(salida) =" +
93         " CURDATE() and estado='FINALIZADO'";
94
95     try (Connection con = ConexionDB.getConexion();
96         PreparedStatement ps = con.prepareStatement(sql)) {
97         try (ResultSet rs = ps.executeQuery()) {
98             if (rs.next()) {
99                 registroDto.setTotalDia(rs.getInt(columnLabel: "totalDia"));
100                 registroDto.setCantidadSalidas(rs.getInt(columnLabel: "cantidadSalidas"));
101                 return registroDto;
102             }
103         }
104     } catch (SQLException e) {
105         registroDto = null;
106     }
107     return registroDto;
108 }

```

registroDTO-función-ingresos Diarios


```

1 package sena.adso.parqueadero.dto;
2
3 import java.time.LocalDate;
4 import java.util.Locale;
5
6 public class RegistroDTO { 4 usages
7     private int totalDia; 4 usages
8     private int CantidadSalidas; 3 usages
9     private final LocalDate fecha = LocalDate.now(); 1 usage
10
11     public RegistroDTO() { 1 usage
12     }
13
14     > public LocalDate getFecha() { return fecha; }
15
16
17
18     ⚡ public int getTotalDia() { no usages
19         return totalDia;
20     }
21
22     > public void setTotalDia(int totalDia) { this.totalDia = totalDia; }
23
24
25     public RegistroDTO(int totalDia, int cantidadSalidas) { no usages
26         this.totalDia = totalDia;
27         CantidadSalidas = cantidadSalidas;
28     }
29
30
31     > public int getCantidadSalidas() { return CantidadSalidas; }
32
33
34     > public void setCantidadSalidas(int cantidadSalidas) { CantidadSalidas = cantidadSalidas; }
35
36
37
38
39     public String toJson() { 1 usage
40         return String.format(Locale.US,
41             format: "{\\"totaldia\\":%d,\\"cantidadsalidas\\":%d}",
42             this.totalDia, this.getCantidadSalidas());
43     }
44 }
45

```

RegistroServlet-validacionUrl-función-ingresos Diarios

```
String pathInfo = req.getPathInfo();
resp.setContentType("application/json;charset=UTF-8");
PrintWriter out = resp.getWriter();

if (pathInfo != null) {
    try {
        out.print(dao.ObtenerIngresoPorDia().toJson());
    } catch (SQLException e) {
        resp.setStatus(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);
        out.print("{\"error\":\"" + e.getMessage() + "\"}");
        return;
    }
} else {
```

reportes.php-consumo-función-ingresos Diarios

```
reportes.php
1  <?php
2
3  require_once 'config.php';
4
5  $reporte = apiRequest('/registros/historial');
6
7  $error = isset($reporte['error']) ? $reporte['error'] : null;
8
9  $totalDia = 0;
10 $cantidadSalidas = 0;
11
12 if (!$error && is_array($reporte)) {
13
14     $totalDia = $reporte['totaldia'];
15     $cantidadSalidas = $reporte['cantidadsalidas'];
16
17 }
18
19 ?>
```

Opción 2: Búsqueda Avanzada en el Historial

Se modificaron las clases RegistroDao.java creando el método listarHistorialFiltrado(String desde, String hasta, String tipo) el cual toma las variables enviadas desde el Servlet y filtrando dependiendo de las enviadas obteniendo una lista de Reportes , además de la clase RegistroServlet.java en la cual se implemento la validación de parámetros tomada de la request y se hizo la llamada de el método creado en DAO y se implemento un formulario de filtración tomando los valores del usuario y enviándolos en la URL para posteriormente tomarlos y enviarlos en el método consumer ala API.

[RegistroDAO.java](#) ← Click aqui para ver

[RegistroServlet.java](#) ← Click aqui para ver

[historial.php](#) ← Click aqui para ver

Imágenes - función 2

vista-función 2-busqueda



Historial de Registros

Fecha Inicio

dd/mm/aaaa



Fecha Fin

dd/mm/aaaa



Tipo Vehículo

Todos



 Filtrar

registroDAO-listarHistorialFiltrado-función 2-busqueda

[ListarHistorialFiltrado\(\)](#) ← Click aqui para ver

registroServlet.java-función 2-busqueda

```

53     if ("FINALIZADO".equalsIgnoreCase(estado)) {
54
55
56         String desde = req.getParameter(s: "desde");
57         String hasta = req.getParameter(s: "hasta");
58         String tipo = req.getParameter(s: "tipo");
59         if (desde != null || hasta != null || tipo != null) {
60
61
62             List<Registro> listaF = dao.listarHistorialFiltrado(desde, hasta, tipo);
63
64             StringBuilder sb = new StringBuilder("[");
65             for (int i = 0; i < listaF.size(); i++) {
66                 if (i > 0) sb.append(",");
67                 sb.append(listaF.get(i).toJson());
68             }
69             sb.append("]");
70             out.print(sb);
71             return;
72         } else {
73
74             lista = dao.listarHistorial();
75         }
76     } else {

```

historial.php-consumer-función 2-busqueda

historial.php

```
1  <?php
2
3  require_once 'config.php';
4
5  $desde = isset($_GET['desde']) ? $_GET['desde'] : '';
6  $hasta = isset($_GET['hasta']) ? $_GET['hasta'] : '';
7  $tipo = isset($_GET['tipo']) ? $_GET['tipo'] : '';
8
9  $query = "/registros?estado=FINALIZADO";
10
11  if (!empty($desde)) {
12      $query .= "&desde=" . urlencode($desde);
13  }
14
15  if (!empty($hasta)) {
16      $query .= "&hasta=" . urlencode($hasta);
17  }
18
19  if (!empty($tipo)) {
20      $query .= "&tipo=" . urlencode($tipo);
21  }
22
23  $historial = apiRequest($query);
24
25  $error = isset($historial['error']) ? $historial['error'] : null;
26
27  $totalRecaudado = 0;
28
29  if (!$error && is_array($historial)) {
30      foreach ($historial as $reg) {
31          $totalRecaudado += $reg['tarifa'];
32      }
33  }
34
35  }
```

```
43     }  
44  
45     ?>
```

Finalmente se anexa el link al repositorio de GitHub:

[Link AppEstacionamineto-integrador <- Click aqui para ver](#)

¡Gracias por leer!